Praktikum JavaScript & TypeScript

Soal Modul 3 - Array dan Method Array

1. Tugas

Kamu bergabung dengan tim backend sebuah marketplace sembako. Data `produk` dan `keranjang` sudah tersedia di bagian `DATA` pada starter.

id	nama	kategori	harga	stok	rating	terjual
P01	Beras Premium 5kg	Sembako	78000	25	4.7	320
P02	Minyak Goreng 2L	Sembako	36000	0	4.5	410
P03	Kopi Bubuk 200g	Minuman	24500	40	4.8	150
P04	Teh Celup 25s	Minuman	9800	60	4.2	95
P05	Sabun Mandi Cair	Perawatan	21000	12	4.4	210
P06	Pasta Gigi 190g	Perawatan	15500	8	4.6	180
P07	Mi Instan Goreng	Sembako	3100	200	4.9	1500
P08	Susu UHT 1L	Minuman	18900	0	4.3	260

Tabel 1: Data `produk`

Isi `keranjang` : P01 sebanyak 1, P07 sebanyak 10, P03 sebanyak 2, P06 sebanyak 1.

> **⚠ Perhatian**
>
> 1. Salin `Soal/starter.js` ke folder `Jawaban/` , misalnya `Jawaban/soal-js3.js` .
> 2. Lengkapi fungsi bertanda `TODO` . Jangan mengubah bagian `DATA` dan `PEMANGGILAN` .
> 3. Jalankan `node Jawaban/soal-js3.js` . Output harus sama persis dengan Contoh Output.
> 4. Soal bertanda **(jelaskan)** dijawab sebagai komentar di file jawaban, di bawah baris `// JAWABAN ...` .
> 5. Kecuali disebut lain, pakai method array (`map` , `filter` , `find` , `some` , `every` , `includes` , `reduce`). Jangan mengubah array `produk` dan `keranjang` .

1.1. Beginner (0.3)

1. Lengkapi `namaProdukTersedia(produk)` : kembalikan array berisi **nama** produk yang stoknya lebih dari 0, dengan urutan sesuai data. Tulis dengan chaining.
2. Lengkapi `totalStok(produk)` memakai `reduce` .
3. Lengkapi `adaKategori(produk, kategori)` dan `semuaRatingDiAtas(produk, batas)` . Fungsi pertama mengecek apakah **ada** produk dalam kategori itu. Fungsi kedua mengecek apakah **semua** produk punya rating minimal `batas` .
4. **(jelaskan)** Temanmu menulis kode berikut dan mendapat error.

   ```
   const kategoriAktif = ["Sembako", "Minuman"];
   console.log(kategoriAktif.contains("Minuman"));
   ```

   Tulis pesan error lengkapnya, jelaskan penyebabnya, lalu tulis versi yang benar. Di mana nama `contains()` benar-benar ada?

1.2. Intermediate (0.3)

1. **Rincian checkout.** Lengkapi `rincianCheckout(keranjang, produk)` yang mengembalikan array string dengan format `Nama xQty = Subtotal`, satu per item keranjang. Lengkapi juga `totalCheckout(keranjang, produk)`. Untuk setiap item keranjang, cari data produknya berdasarkan `idProduk`.
2. **Kode temanmu.** `totalTerjual` seharusnya menjumlahkan kolom `terjual` semua produk, tetapi outputnya `[object Object]410150...`. **(jelaskan)** mengapa hasilnya string aneh, bukan angka dan bukan error. Lalu perbaiki.
3. **Kode temanmu.** `urutHargaTermurah` dipakai untuk menampilkan tiga produk termurah. Hasilnya benar, tetapi setelah fungsi dipanggil, `produk[0]` berubah menjadi "Mi Instan Goreng". **(jelaskan)** mengapa data asli ikut berubah, dan mengapa hal ini berbahaya di aplikasi nyata. Lalu perbaiki agar data asli tidak berubah.
4. **Kode temanmu.** `cariProdukHabis` seharusnya mengembalikan nama produk **pertama** (urutan data) yang stoknya 0, atau `null` jika tidak ada. Kerjakan soal ini **setelah** nomor 3 diperbaiki.
   a. **(jelaskan)** Setelah nomor 3 diperbaiki, fungsi ini mengembalikan "Susu UHT 1L", bukan "Minyak Goreng 2L". Mengapa `return` di dalam `forEach` tidak menghentikan perulangan?
   b. **(jelaskan)** Sebelum nomor 3 diperbaiki, fungsi ini justru menghasilkan "Minyak Goreng 2L". Mengapa bisa kebetulan benar?
   c. Perbaiki fungsi dengan method yang tepat, tanpa `forEach` dan tanpa `for`.

1.3. Advanced (0.4)

1. **Laporan per kategori.** Lengkapi `laporanKategori(produk)` yang mengembalikan satu string per kategori, dengan urutan kategori sesuai kemunculan pertamanya di data. Format: `Kategori: N produk, stok S, omzet O`, dengan omzet = jumlah dari `harga × terjual`. Jangan menulis nama kategori secara langsung di kode; daftar kategori harus didapat dari data. Boleh memakai `for...of` untuk mengumpulkan daftar kategori.
2. **Checkout lengkap.**
   a. Lengkapi `checkout(keranjang, produk, kodePromo)`. Fungsi mengembalikan string. Cek secara berurutan:
      a. Keranjang kosong: `GAGAL: keranjang kosong`
      b. Untuk setiap item, sesuai urutan keranjang: produk tidak ada di data: `GAGAL: produk ID tidak ditemukan`; qty melebihi stok: `GAGAL: stok NAMA tidak cukup (sisa S)`
      c. `kodePromo` bukan `null`, `"HEMAT10"`, atau `"GRATIS-ONGKIR"`: `GAGAL: kode promo tidak valid`

      Jika semua lolos, hitung:
      - `HEMAT10`: diskon 10% dari subtotal, dibulatkan ke bawah, maksimal Rp 20.000.
      - Ongkir Rp 15.000 jika subtotal di bawah Rp 100.000, selain itu gratis. Kode `GRATIS-ONGKIR` membuat ongkir selalu 0.
      - Kembalikan `Subtotal: X, Diskon: D, Ongkir: O, Total: T`.
   b. **(jelaskan)** Untuk mencari produk, mengapa `find` lebih tepat daripada `filter(...)[0]`? Jika keranjang berisi 50 item dan data berisi 10.000 produk, kira-kira berapa kali perbandingan id dilakukan? Usulkan cara agar pencarian lebih cepat. Tidak perlu ditulis kodenya; konsep ini muncul lagi di Modul 6.

2. Contoh Output

Contoh Output

```
== Beginner 1 ==
Beras Premium 5kg, Kopi Bubuk 200g, Teh Celup 25s, Sabun Mandi Cair, Pasta Gigi 190g, Mi Instan
Goreng
== Beginner 2 ==
345
== Beginner 3 ==
true false
```

```
true false
== Intermediate 1 ==
Beras Premium 5kg x1 = 78000
Mi Instan Goreng x10 = 31000
Kopi Bubuk 200g x2 = 49000
Pasta Gigi 190g x1 = 15500
Total: 173500
== Intermediate 2 ==
3125
== Intermediate 3 ==
Mi Instan Goreng, Teh Celup 25s, Pasta Gigi 190g
Produk pertama di data asli: Beras Premium 5kg
== Intermediate 4 ==
Minyak Goreng 2L
null
== Advanced 1 ==
Sembako: 3 produk, stok 225, omzet 44370000
Minuman: 3 produk, stok 100, omzet 9520000
Perawatan: 2 produk, stok 20, omzet 7200000
```

Contoh Output

```
== Advanced 2 ==
Subtotal: 173500, Diskon: 0, Ongkir: 0, Total: 173500
Subtotal: 173500, Diskon: 17350, Ongkir: 0, Total: 156150
Subtotal: 44100, Diskon: 0, Ongkir: 0, Total: 44100
Subtotal: 44100, Diskon: 0, Ongkir: 15000, Total: 59100
Subtotal: 234000, Diskon: 20000, Ongkir: 0, Total: 214000
GAGAL: stok Pasta Gigi 190g tidak cukup (sisa 8)
GAGAL: stok Minyak Goreng 2L tidak cukup (sisa 0)
GAGAL: produk P99 tidak ditemukan
GAGAL: kode promo tidak valid
GAGAL: keranjang kosong
```

3. Pengumpulan

⚠ Perhatian

- Kode soal: `Jawaban/soal-js3.js`. Jawaban **(jelaskan)** ditulis sebagai komentar di bawah baris `// JAWABAN ...` yang sesuai.
- Coba Sendiri dari HO dikerjakan di fungsi `cobaSendiri()` pada file yang sama. Jalankan dengan `node Jawaban/soal-js3.js coba`.
- Jalankan `npm run selesai -- --durasi "..." --macet "..."`, lalu minta asisten memeriksa.